

Flutter Navigation

Navigation in Flutter

Eng | [Ruqaih Hussein Salman](#)

2027-2028

Table of Contents

Complete lecture structure

Section	Coverage	Page
1	Navigation mental model, route stack, decision map, and BuildContext	3
2	push, pop, replacement, reset, pop-until, and PopScope back handling	5
3	Send data through constructors and RouteSettings	8
4	Return typed results with await Navigator.push and pop(result)	11
5	Named routes, onGenerateRoute, and modern Router/go_router preview	11
6	Hero shared-element animation, asset setup, and custompage transitions	14
7	TabBar and state-preserving Material 3 NavigationBar destinations	16
8	AlertDialog, SimpleDialog, show ModalBottomSheet, and typed modal results	19
9	show TimePicker and show DatePicker	22
10	image_picker setup, gallery/camera selection, and lifecycle notes	23
11	CourseHub integrated lab, NavigatorObserver, testing, and debugging	26
12	Decision guide, applied assessment, and official references	31
13	Applied Project Pack: specification, milestones, tests, rubric, and final checklist	33

Learning Outcomes and Study Map

What the student should master by the end of the lecture

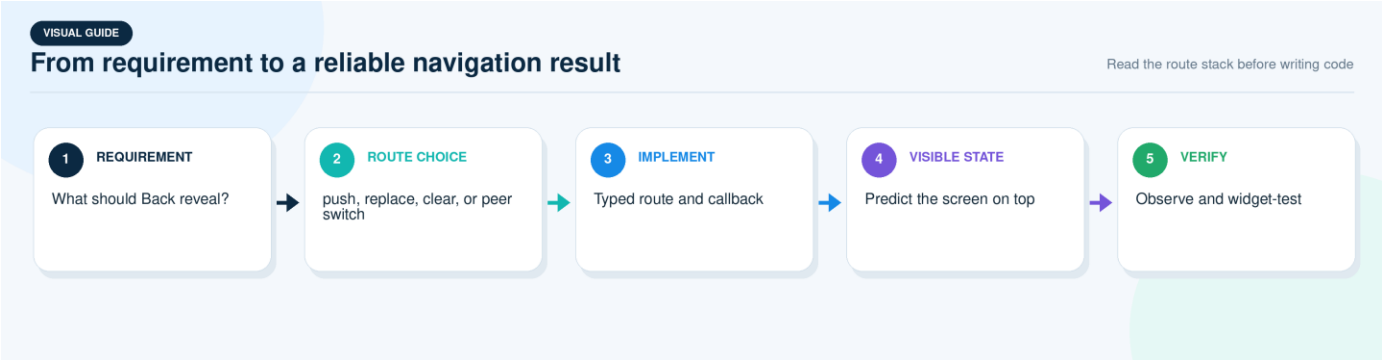
Area	Student mastery
Mental model	Explain routes, Navigator, stack history, context, and visible navigation feedback.
Forward navigation	Open a route with MaterialPageRoute and choose push, replacement, or reset correctly.
Data flow	Pass typed data to a destination and return a typed result to the source screen.
Route organization	Use named routes for learning/legacy code and explain why they are not the default choice for most new apps.
Destinations	Build synchronized TabBar/TabBarView and state-preserving NavigationBar layouts.
Modal routes	Await dialog, date, time, and selection results safely, including cancellation.
Media selection	Use the current image_picker XFile API and present gallery/camera outcomes.
Reliability	Check context.mounted, prevent duplicate pushes, handle back navigation, and verify the route stack.
Observation	Log push, pop, and replacement events with NavigatorObserver.
Verification	Write a widget test that opens a destination and returns to the source.
Integration	Connect destination navigation, typed data, modal results, and back protection in CourseHub.

LEARNING STRATEGY

- Predict the route stack before running each callback; then compare the visible screen with the prediction.
- For every navigation action, identify the source route, destination route, data sent, result returned, and history that remains.
- When navigation fails, inspect the callback, BuildContext, route construction, stack method, and visible feedback—in that order.

Visual Roadmap

From a navigation requirement to a reliable result



Standard	Target
Compatibility	Flutter 3.44.x • Dart 3.12.x • Material 3
Core scope	Navigator 1.0 imperative navigation for mobile learning examples; Router/go_router is introduced as the scalable alternative.
Example scope	Focused output panels show only the widgets and states created by the displayed snippet. Framework dialogs/pickers appear only when the snippet invokes them; unrelated app chrome is omitted.
Output standard	Focused output panels show only widgets and states created by the displayed snippet. No decorative cards, lists, AppBars, or controls are added unless they appear in code.
External package	image_picker 1.2.3, using XFile and pickImage rather than removed getImage/PickedFile APIs.

SNIPPET CONTRACT

- Each focused example teaches one navigation principle rather than pretending to be a complete application.
- Build fragments are labeled explicitly; only Section 11.1 is a complete copy-and-run application.
- Expected outputs contain only UI created by the shown code plus framework surfaces directly invoked by it.
- Captions state any assumed surrounding context. Diagrams are labeled as diagrams, not application screenshots.

1. Flutter Navigation Mental Model

Routes, Navigator, stack history, callbacks, and visible screens

LECTURE 7 • MAP 1

Navigation = Route Stack + History

The API choice is correct only when it leaves the Back history the workflow needs.

20-SECOND QUESTION

What is visible now, and what should the Back action reveal?

THE STACK CHANGES IN THREE PREDICTABLE STATES

1

Start at Home

Home is the only route. The top route is visible.

Home

2

push Details

A new route is added above Home. Home stays in history.

Home

Details

3

pop with result

Details is removed. Home is visible and the caller Future completes.

Home

CONNECTION

Route describes one history entry; Navigator changes the stack; Future<T?> completes when the pushed route pops.

CHOOSE THE HISTORY YOU WANT

push

Keep the current route below the new destination.

pushReplacement

Replace only the current route.

pushAndRemoveUntil

Push a destination and clear earlier history.

PopScope

Guard a Back attempt before the route is removed.

REMEMBER THESE FOUR ANCHORS

Route

One entry in navigation history.

Navigator

Owens and changes the route stack.

BuildContext

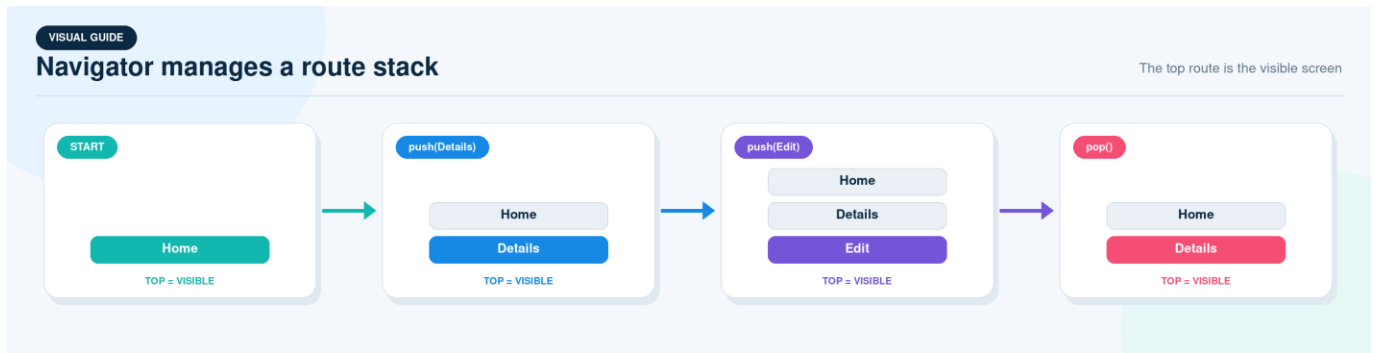
Finds the relevant Navigator.

Future<T?>

Completes when the pushed route pops.

QUICK IDEA

- A route is a screen represented by a widget. Navigator displays routes as a stack and changes that stack through push and pop operations.
- Navigation is not complete until the callback changes the route stack and the user sees the new route, returned result, or cancellation state.



push adds a route above the current route; pop removes the top route and reveals the route below it.

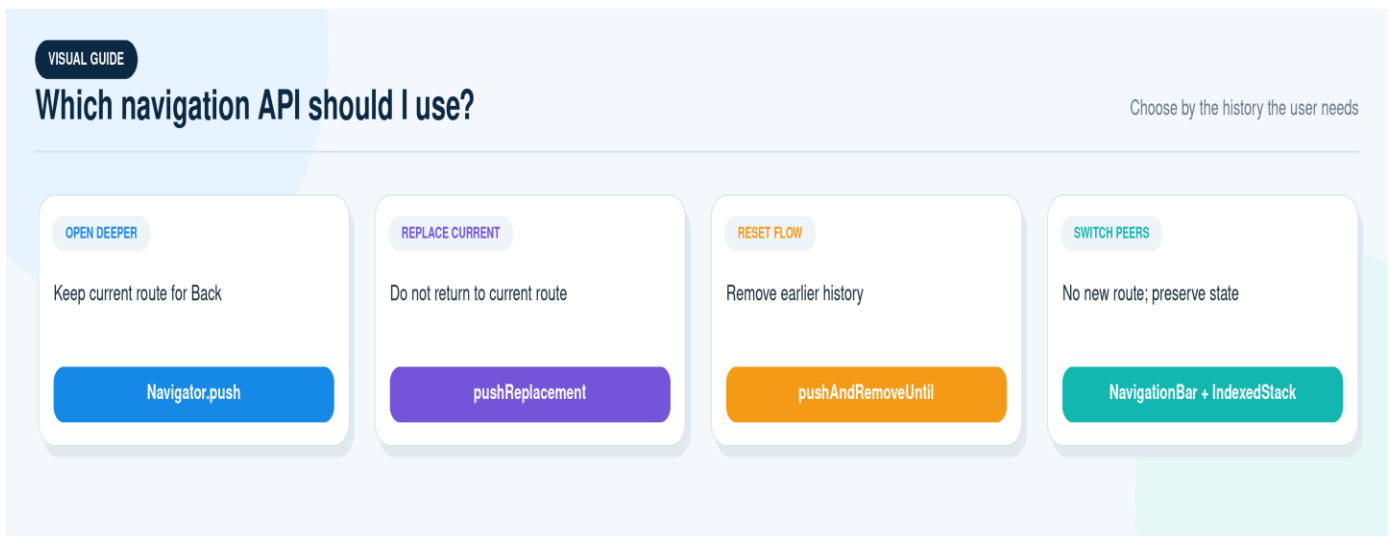
Term	Meaning	Diagnostic question
Route	A page/screen represented by a widget and wrapped in a Route object.	Which screen should be created?
Navigator	The widget/state that manages route history.	Which Navigator encloses this context?
Route stack	The ordered history of active routes.	Which routes should remain after this action?
BuildContext	Locates the nearest Navigator and inherited configuration.	Is this context below MaterialApp/Navigator?
Future result	The value completed when a pushed route later pops.	What type can the destination return?

REMEMBER

- push = add a route; pop = remove the top route; await push = wait for the value supplied to pop(result).
- The app bar back button and Android system back normally call pop on the current route.

1.1. Navigation Decision Map

Choose the API from the required history, returned value, and visual surface



REMEMBER

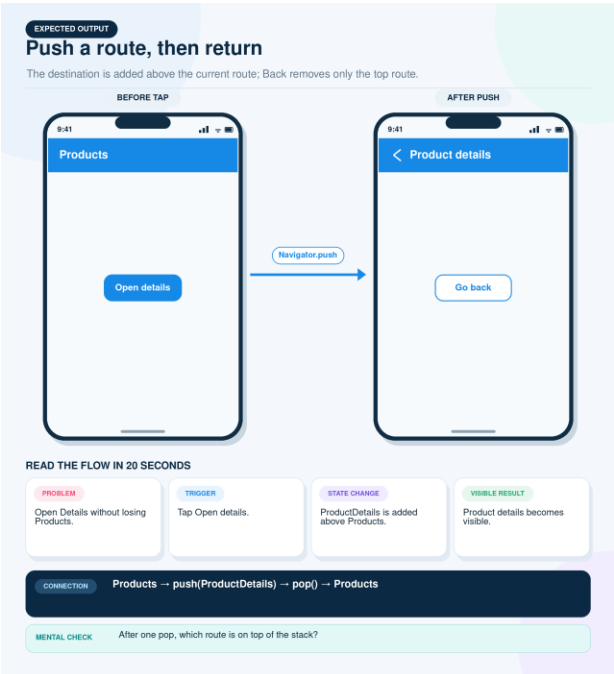
- The best API is the smallest one that leaves the correct Back history.
- A choice-producing modal route should return a value; a full-screen route should represent a meaningful workflow step.

2. Navigator.push, pop, Replacement, and Back Handling

Choose the stack operation that matches the workflow

2.1. Navigate to a New Screen and Back

MaterialPageRoute + Navigator.push + Navigator.pop

Code	Expected output state(s)
<pre>FOCUSED PUSH + POP FRAGMENTS // Source route return Scaffold(appBar: AppBar(title: const Text('Products')), body: Center(child: FilledButton(onPressed: () => Navigator.push<void>(context, MaterialPageRoute<void>(builder: (_) => const ProductDetailsScreen(),),), child: const Text('Open details'),),); // Destination route class ProductDetailsScreen extends StatelessWidget { const ProductDetailsScreen({super.key}); @override Widget build(BuildContext context) { return Scaffold(appBar: AppBar(title: const Text('Product details')), body: Center(child: OutlinedButton(onPressed: () => Navigator.pop(context), child: const Text('Go back'),),),); }</pre>	 EXPECTED OUTPUT Push a route, then return The destination is added above the current route; Back removes only the top route. BEFORE TAP Products screen with 'Open details' button. AFTER PUSH Product details screen with 'Go back' button. READ THE FLOW IN 20 SECONDS PROBLEM: Open Details without losing Products. TRIGGER: Tap Open details. STATE CHANGE: ProductDetails is added above Products. VISIBLE RESULT: Product details becomes visible. CONNECTION: Products -- push(ProductDetails) -- pop() --> Products MENTAL CHECK: After one pop, which route is on top of the stack? The two build fragments create exactly these routes: Products before push and Product details after push. TRY IT Add a third route and predict the stack after two pushes and one pop.

KEY NOTES

- Use MaterialPageRoute<void> when the destination does not return a value; use a concrete result type such as MaterialPageRoute<String> when it does.
- Prefer const constructors for routes whose inputs are immutable.
- Do not call navigation directly from build(); connect it to a callback, lifecycle decision, or listener with duplicate-call protection.

2.2. Replacement, Reset, and Pop-Until Methods

Preserve only the route history the user should be able to revisit

VISUAL GUIDE

Choose the method by the history you want

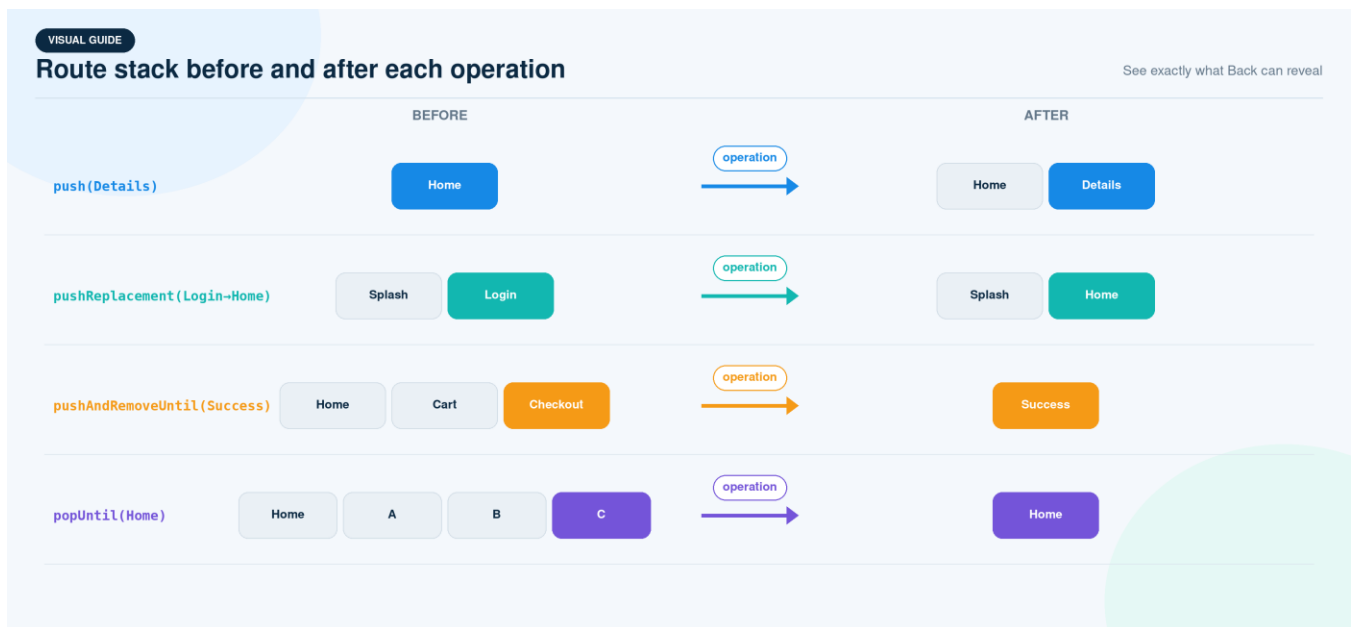
API names become obvious after the stack decision

push	Keep the current route below the new one Home → Details
pushReplacement	Replace only the current route Login → Home
pushAndRemoveUntil	Push and clear selected earlier routes Checkout → Success
popUntil	Pop repeatedly until a route matches Deep page → Home

Method	History effect	Typical use
push	Adds the new route above the current route.	Open details and allow Back.
pushReplacement	Adds a new route and removes the current route after transition.	Login → Home; Splash → Onboarding.
pushAndRemoveUntil	Pushes a route, then removes earlier routes until the predicate matches.	Checkout success; logout reset.
popUntil	Pops repeatedly until a route matches.	Return from a deep workflow to Home.
canPop / maybePop	Checks or attempts to pop without assuming history exists.	Custom back controls and nested navigators.

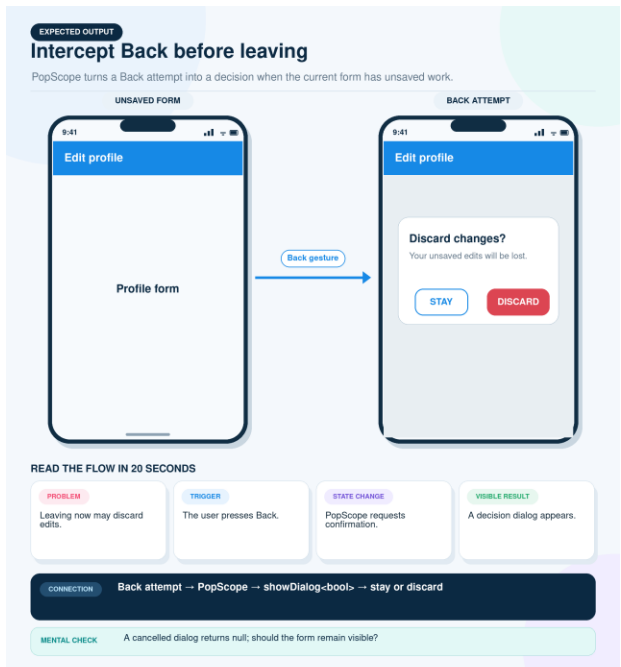
FOCUSED CODE

- `Navigator.pushReplacement<void, void>(context, MaterialPageRoute(builder: (_) => const HomeScreen()));`
- `Navigator.pushAndRemoveUntil<void>(context, MaterialPageRoute(builder: (_) => const SuccessScreen()), (route) => false);`
- `Navigator.popUntil(context, ModalRoute.withName('/'));`



2.3. Protect Unsavd Work with PopScope

Intercept a back attempt, confirm intent, then allow the route to close

Code	Expected output state(s)
<pre>POPSCOPE GUARD + DIALOG + SCREEN bool hasChanges = true; bool dialogOpen = false; Future<void> handleBack() async { if (dialogOpen) return; dialogOpen = true; final discard = await showDialog<bool>(context: context, builder: (dialogContext) => AlertDialog(title: const Text('Discard changes?'), content: const Text('Your unsaved edits will be lost.'), actions: [TextButton(onPressed: () => Navigator.pop(dialogContext, false), child: const Text('Stay'),), FilledButton(onPressed: () => Navigator.pop(dialogContext, true), child: const Text('Discard'),),],),); dialogOpen = false; if (discard != true !mounted) return; setState(() => hasChanges = false); WidgetsBinding.instance.addPostFrameCallback((_) { if (mounted) Navigator.pop(context); }); } return PopScope<void>(canPop: !hasChanges, onPopInvokedWithResult: (didPop, result) { if (!didPop) handleBack(); }, child: Scaffold(appBar: AppBar(title: const Text('Edit profile')), body: const Center(child: Text('Profile form')),),);</pre>	 The snippet creates <i>Edit profile</i> and the confirmation dialog. <i>Stay</i> keeps the route; <i>Discard</i> enables and completes the pop. TRY IT Set <i>hasChanges</i> to false and verify that <i>Back</i> closes directly.

COMMON MISTAKES

- Calling `Navigator.pop` after confirmation while `canPop` remains false, causing the second pop to be blocked again.
- Using the deprecated `onPopInvoked` callback instead of `onPopInvokedWithResult` in current Flutter code.
- Showing a confirmation dialog for every back action even when no data changed.

3. Send Data to a Destination Screen

Prefer typed constructor parameters for direct route creation

LECTURE 7 • MAP 2

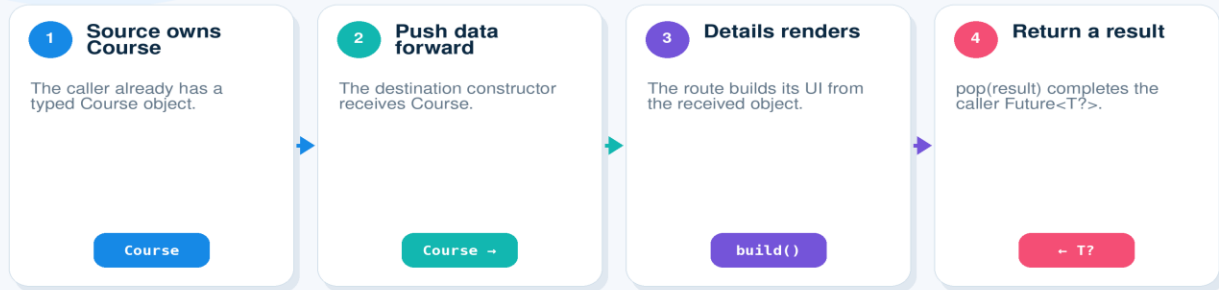
Data Forward • Result Back

Treat navigation as two typed contracts: input to the destination and an optional result to the caller.

20-SECOND QUESTION

What travels forward, what comes back, and who builds the route?

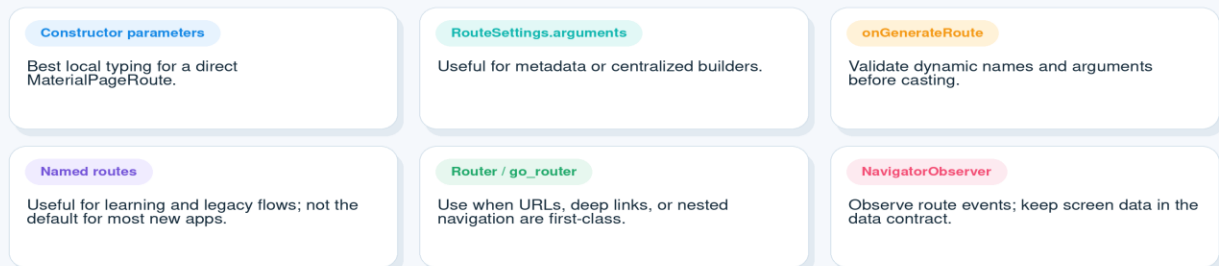
ONE USER ACTION, TWO DATA DIRECTIONS



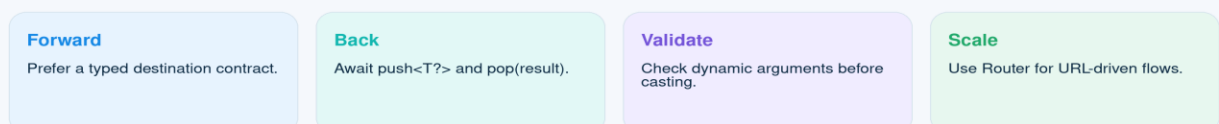
CONNECTION

Forward: typed constructor or validated `RouteSettings.arguments`. Back: `await push<T?>` plus `Navigator.pop(result)`.

HOW SHOULD THE DESTINATION BE CREATED?



REMEMBER THESE FOUR ANCHORS



3.1. Pass a Model through the Destination Constructor

The destination declares exactly what it needs

Code	Expected output state(s)
<pre> TYPED MODEL + SOURCE + DESTINATION class Course { const Course(this.title, this.summary); final String title; final String summary; } const course = Course('Flutter Navigation', 'Routes, data flow, tabs, dialogs, and pickers.',); // Source route build return Scaffold(appBar: AppBar(title: const Text('Courses')), body: ListTile(title: Text(course.title), subtitle: const Text('Lecture 7'), onTap: () => Navigator.push<void>(context, MaterialPageRoute<void>(builder: (_) => const CourseDetailsScreen(course),),),),); class CourseDetailsScreen extends StatelessWidget { const CourseDetailsScreen(this.course, {super.key}); final Course course; @override Widget build(BuildContext context) => Scaffold(appBar: AppBar(title: Text(course.title)), body: Padding(padding: const EdgeInsets.all(16), child: Text(course.summary),),); } </pre>	EXPECTED OUTPUT Pass typed data to a destination The caller already owns a <code>Course</code> object and forwards it through the destination constructor. READ THE FLOW IN 20 SECONDS PROBLEM Details needs the selected <code>Course</code>. TRIGGER Tap the course tile. STATE CHANGE A typed <code>Course</code> enters the new route. VISIBLE RESULT The details screen renders its data. CONNECTION <code>CourseList --> MaterialPageRoute(Course) --> CourseDetails</code> MENTAL CHECK What compile-time type should the destination constructor require? The source <code>ListTile</code> shows the same <code>Course</code> data that the destination receives through its constructor. TRY IT Add <code>durationMinutes</code> to <code>Course</code> and display it on the destination.

KEY NOTES

- Constructor parameters are explicit, typed, testable, and easy to trace through the widget tree.
- Do not pass a `BuildContext`, controller, or mutable screen State when a small immutable model is sufficient.
- For large objects, pass an identifier and let the destination load fresh data when that better matches the architecture.

3.2. RouteSettings and ModalRoute Arguments

An alternative when route construction or metadata is centralized

Approach	How data is supplied	Best fit
Constructor	<code>CourseDetailsScreen(course: course)</code>	Direct <code>MaterialPageRoute</code> creation; strongest local typing.
RouteSettings	<code>MaterialPageRoute(settings: RouteSettings(arguments: course), ...)</code>	Route metadata, analytics, or route observers.
Named arguments	<code>Navigator.pushNamed(arguments: course)</code>	Teaching/legacy named-route flows.
<code>onGenerateRoute</code>	Read <code>settings.arguments</code> and construct the route centrally.	Central route validation and unknown-route handling.

Code

ROUTESETTINGS CALLER + DESTINATION

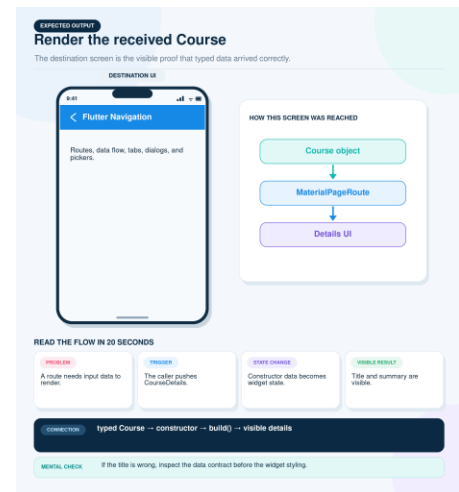
```
Navigator.push<void>(
  context,
  MaterialPageRoute<void>(
    settings: RouteSettings(arguments: course),
    builder: (_) => const RouteSettingsCourseScreen(),
  ),
);

class RouteSettingsCourseScreen extends StatelessWidget {
  const RouteSettingsCourseScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final course = ModalRoute.of(context)!
      .settings.arguments as Course;

    return Scaffold(
      appBar: AppBar(title: Text(course.title)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Text(course.summary),
      ),
    );
  }
}
```

Expected output state(s)



The snippet reads *Course* from *settings.arguments* and renders exactly this destination.

TRY IT Remove arguments and inspect why centralized routes should validate the type.

REMEMBER

- Prefer constructor parameters for direct routes; use *RouteSettings* when the value belongs to route metadata or centralized construction.
- The unchecked cast in this focused example is safe only when the caller is controlled. Validate the type in *onGenerateRoute* otherwise.
- The visible destination is the same as Section 3.1; only the data channel changes.

4. Return Data from a Screen

await the pushed route, then complete it with `Navigator.pop(result)`

4.1. Typed Selection Result

The destination returns one value to the caller

Code	Expected output state(s)				
<pre>TYPED RESULT: CALLER + DESTINATION BUILD Future<void> chooseTheme() async { final theme = await Navigator.push<String>(context, MaterialPageRoute<String>(builder: (_) => const ThemePickerScreen(),),); if (theme == null !mounted) return; ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Selected theme: \$theme')),); } // Caller widget FilledButton(onPressed: chooseTheme, child: const Text('Choose theme'),); // ThemePickerScreen.build return Scaffold(appBar: AppBar(title: const Text('Choose a theme')), body: ListView(children: [for (final theme in ['Ocean', 'Violet', 'Amber']) ListTile(title: Text(theme), onTap: () => Navigator.pop(context, theme),),],),);</pre>	EXPECTED OUTPUT Return a typed result to the caller The pushed route completes the caller's awaited Future when it pops with a value. <pre>graph LR Caller[CALLER] -- "await push<String>" --> ChooseValue[CHOOSE VALUE] ChooseValue -- "pop('Ocean')" --> ResultReceived[RESULT RECEIVED]</pre> READ THE FLOW IN 20 SECONDS <table><tr><td>PROBLEM The caller needs a theme choice.</td><td>TRIGGER Open the choice screen.</td><td>STATE CHANGE pop returns Ocean through Future<String?>.</td><td>VISIBLE RESULT The caller shows the selected theme.</td></tr></table> CONNECTION <code>await Navigator.push<String>() -- Navigator.pop('Ocean')</code> MENTAL CHECK If the user cancels, what nullable value reaches the caller? The caller opens ThemePickerScreen. Choosing Ocean completes Future<String> and displays Selected theme: Ocean. TRY IT Dismiss the destination with system Back and verify that the result is null.	PROBLEM The caller needs a theme choice.	TRIGGER Open the choice screen.	STATE CHANGE pop returns Ocean through Future<String?>.	VISIBLE RESULT The caller shows the selected theme.
PROBLEM The caller needs a theme choice.	TRIGGER Open the choice screen.	STATE CHANGE pop returns Ocean through Future<String?>.	VISIBLE RESULT The caller shows the selected theme.		

5. Named Routes and Central Route Generation

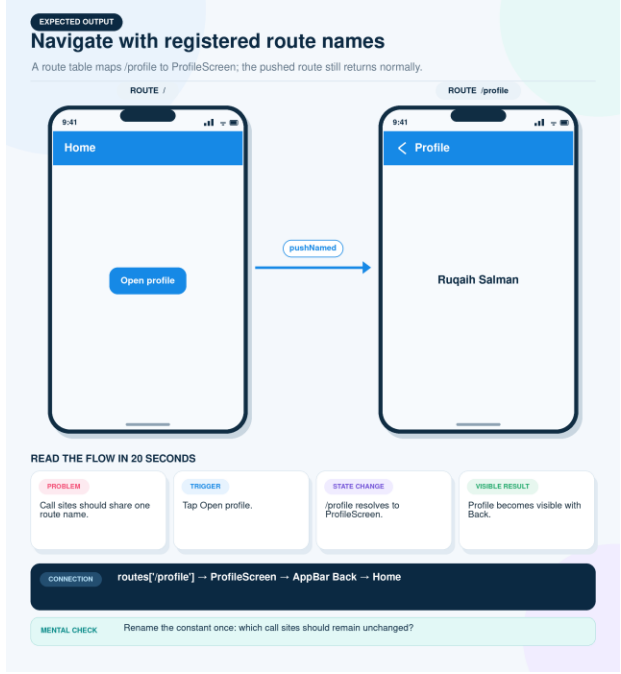
Useful for learning and legacy projects, but not the default recommendation for most new apps

CURRENT GUIDANCE

- Flutter documentation no longer recommends named routes for most applications because of deep-link and browser-history limitations.
- For small apps, prefer Navigator with MaterialPageRoute. For advanced deep linking and web URL synchronization, use Router or a routing package such as go_router.

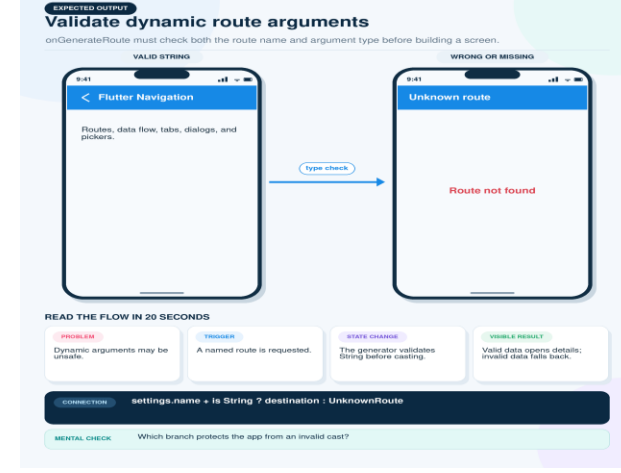
5.1. Basic Named-Route Pattern

Register a name, push it, then pop normally

Code	Expected output state(s)
<pre> MINIMAL NAMED-ROUTE PATTERN MaterialApp(initialRoute: '/', routes: { '/': (_) => const HomeScreen(), ProfileScreen.routeName: (_) => const ProfileScreen(), },); class HomeScreen extends StatelessWidget { const HomeScreen({super.key}); @override Widget build(BuildContext context) => Scaffold(appBar: AppBar(title: const Text('Home')), body: Center(child: FilledButton(onPressed: () => Navigator.pushNamed(context, ProfileScreen.routeName,), child: const Text('Open profile'),),),); } class ProfileScreen extends StatelessWidget { const ProfileScreen({super.key}); static const routeName = '/profile'; @override Widget build(BuildContext context) => Scaffold(appBar: AppBar(title: const Text('Profile')), body: const Center(child: Text('Ruqaih Salman')),); } </pre>	 EXPECTED OUTPUT Navigate with registered route names A route table maps /profile to ProfileScreen; the pushed route still returns normally. READ THE FLOW IN 20 SECONDS PROBLEM: Call sites should share one route name. TRIGGER: Tap Open profile. STATE CHANGE: /profile resolves to ProfileScreen. VISIBLE RESULT: Profile becomes visible with Back. CONNECTION: routes['profile'] -> ProfileScreen -> AppBar Back -> Home MENTAL CHECK: Rename the constant once; which call sites should remain unchanged? The / route shows Open profile; /profile shows Ruqaih Salman and returns with the AppBar Back action. TRY IT Rename /profile through ProfileScreen.routeName and change no other call site.

5.2. Validate Arguments with onGenerateRoute

Centralize route construction and provide an unknown-route fallback

Code	Expected output state(s)
<pre> TYPED ARGUMENT CHECK IN onGenerateRoute Route<dynamic> buildRoute(RouteSettings settings) { if (settings.name == '/course' && settings.arguments is String) { final title = settings.arguments! as String; return MaterialPageRoute<void>(settings: settings, builder: (_) => Scaffold(appBar: AppBar(title: Text(title)), body: const Padding(padding: EdgeInsets.all(16), child: Text('Routes, data flow, tabs, dialogs, and pickers.',)),),),); } return MaterialPageRoute<void>(settings: settings, builder: (_) => Scaffold(appBar: AppBar(title: const Text('Unknown route')), body: const Center(child: Text('Route not found')),),); } // MaterialApp configuration onGenerateRoute: buildRoute, </pre>	 EXPECTED OUTPUT Validate dynamic route arguments onGenerateRoute must check both the route name and argument type before building a screen. READ THE FLOW IN 20 SECONDS PROBLEM: Routes, data flow, tabs, dialogs, and pickers. TRIGGER: A named route is requested. STATE CHANGE: The generator validates String before casting. VISIBLE RESULT: Valid data opens details; invalid data falls back. CONNECTION: settings.name is String ? destination: UnknownRoute MENTAL CHECK: Which branch protects the app from an invalid cast? A valid String builds the course screen; a wrong or missing argument builds the controlled fallback. TRY IT Call /course with an int and verify that the fallback screen appears.

Code	Expected output state(s)
<pre>// Call from a callback below that MaterialApp Navigator.pushNamed(context, '/course', arguments: 'Flutter Navigation',);</pre>	

5.3. Modern Routing Preview

Use Navigator for the stack mental model; use Router/go_router when URLs and deep links become first-class requirements

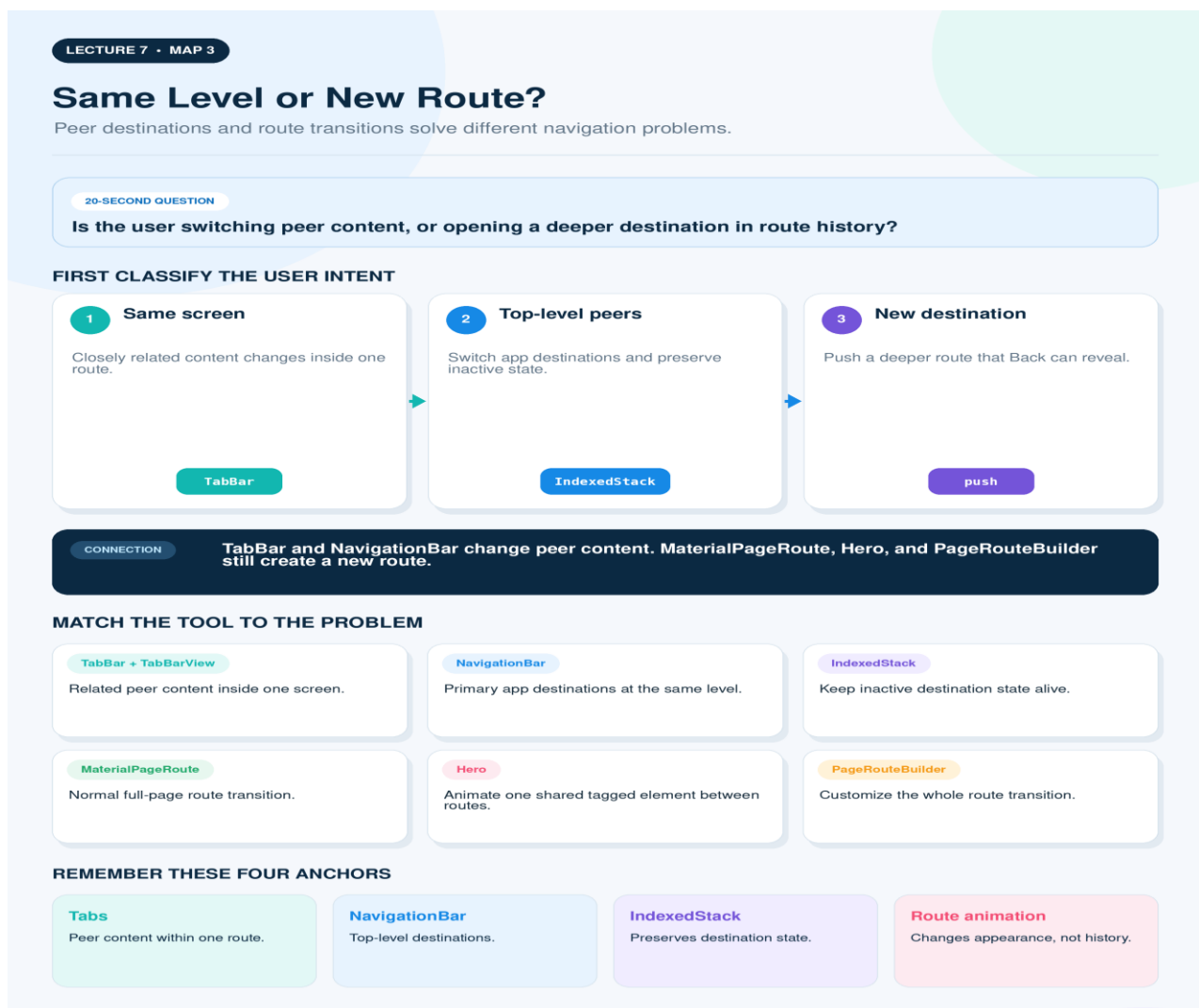
Requirement	Navigator 1.0	Router / go_router
Small mobile flow	Excellent starting point	Usually unnecessary
Deep links	Manual coordination	Designed for URL-driven navigation
Flutter web history	Limited manual handling	Synchronizes browser URL/history
Nested shell routes	Possible but verbose	Declarative route hierarchy
Teaching route stack	Best option	Introduce after the mental model

FOCUSED PREVIEW

- Navigator is still the clearest way to learn push, pop, and typed results.
- A scalable app can later move route configuration to Router or go_router without changing the underlying history concepts.

6. Hero Shared-Element Navigation

Connect the same visual object across two routes



6.1. Animate an Image between List and Detail Screens

Matching tags create the shared-element transition

REQUIRED ASSET SETUP

- Create assets/course.png inside the project.
- Register it in pubspec.yaml under flutter: assets: - assets/course.png.
- Use the same Hero tag on both active routes.

Code

MATCHING HERO TAGS ON BOTH ROUTES

```
const heroTag = 'course-image';

// Source route
return Scaffold(
  appBar: AppBar(title: const Text('Gallery')),
  body: Center(
    child: InkWell(
      onTap: () => Navigator.push<void>(
        context,
        MaterialPageRoute<void>(
          builder: (_) => const PhotoScreen(),
        ),
      ),
    child: const Hero(
      tag: heroTag,
      child: Image(
        image: AssetImage('assets/course.png'),
        width: 160,
      ),
    ),
  ),
);

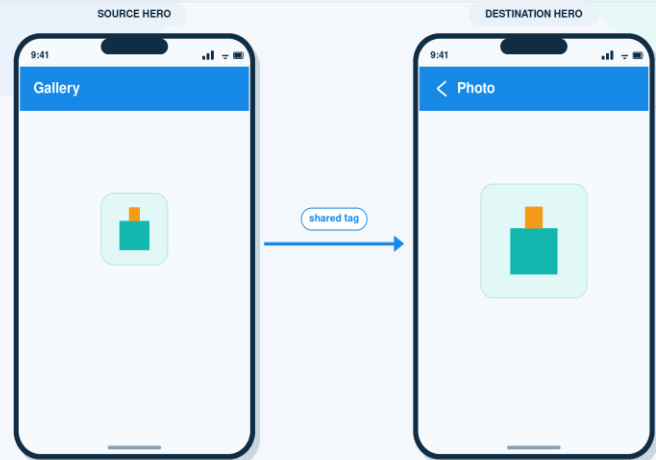
// PhotoScreen.build
return Scaffold(
  appBar: AppBar(title: const Text('Photo')),
  body: const Center(
    child: Hero(
      tag: heroTag,
      child: Image(
        image: AssetImage('assets/course.png'),
        width: 320,
      ),
    ),
  ),
);
```

Expected output state(s)

EXPECTED OUTPUT

Animate one shared element with Hero

The same tagged element changes size and position while the route itself is pushed normally.



READ THE FLOW IN 20 SECONDS

PROBLEM

The image should feel continuous across routes.

TRIGGER

Tap the thumbnail.

STATE CHANGE

Hero matches identical tags.

VISIBLE RESULT

The image expands into Photo.

CONNECTION

Hero(tag: 'course-photo') on both routes

MENTAL CHECK

Hero animates the shared element; which API still manages route history?

Both routes show the same asset with the same Hero tag; only its size and route position change.

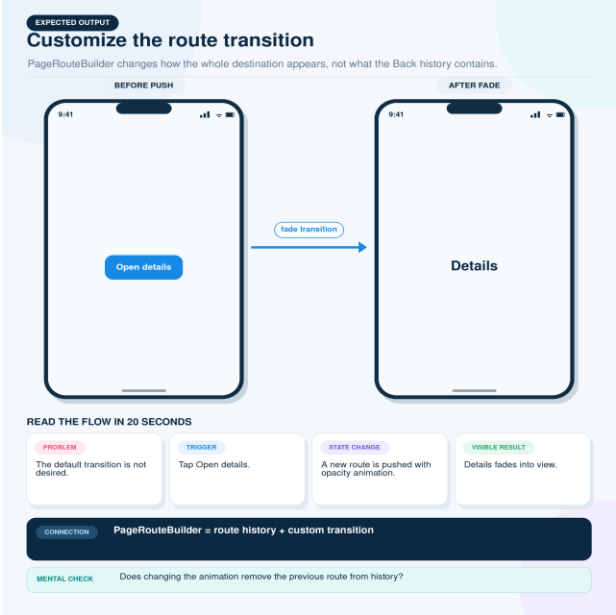
TRY IT Change one tag and observe that the shared-element transition no longer connects.

COMMON MISTAKES

- Using different tag values on the source and destination.
- Creating duplicate Hero tags in the same route subtree.
- Animating widgets with unrelated shapes or clipping without testing the in-flight appearance.

6.2. Custom Page Transition with PageRouteBuilder

Use a route transition when the entire page should fade or slide; use Hero for one shared visual element

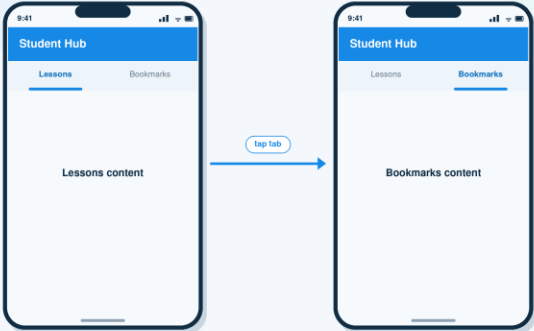
Code	Expected output state(s)
<pre>FADE THE WHOLE DESTINATION ROUTE FilledButton(onPressed: () => Navigator.push<void>(context, MaterialPageRoute<void>(pageBuilder: (_, animation, __) => const Scaffold(body: Center(child: Text('Details')),), transitionsBuilder: (_, animation, __, child) { return FadeTransition(opacity: CurvedAnimation(parent: animation, curve: Curves.easeInOut,), child: child,); },), child: const Text('Open details'),);</pre>	 The transition fades between exactly these two states; the snippet adds no AppBar or unrelated controls. TRY IT Replace <code>FadeTransition</code> with <code>SlideTransition</code> and keep the same destination.
CHOOSING THE ANIMATION • <code>MaterialPageRoute</code>: platform-appropriate full-page transition. • <code>Hero</code>: one matching widget changes position and size across routes. • <code>PageRouteBuilder</code>: custom animation for the whole destination route.	

7. Destination Navigation: Tabs and NavigationBar

Switch between peer destinations without building a deep route stack

7.1. TabBar with DefaultTabController

Keep TabBar, TabBarView, and controller length synchronized

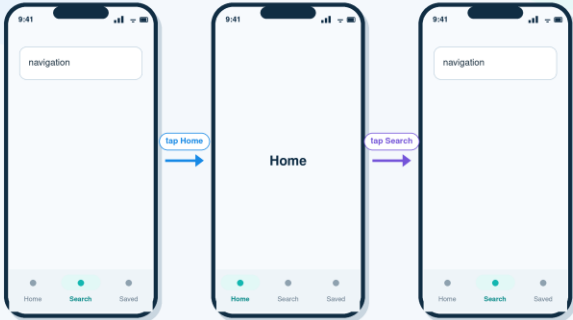
Code	Expected output state(s)				
<pre>TABBAR + TABBARVIEW return DefaultTabController(length: 2, child: Scaffold(appBar: AppBar(title: const Text('Student Hub'), bottom: const TabBar(tabs: [Tab(text: 'Lessons'), Tab(text: 'Bookmarks'),],),), body: const TabBarView(children: [Center(child: Text('Lessons content')), Center(child: Text('Bookmarks content')),],),),);</pre>	EXPECTED OUTPUT Switch tabs inside one route TabBar changes peer content without pushing a deeper route onto the Navigator stack. LESSONS TAB BOOKMARKS TAB  READ THE FLOW IN 20 SECONDS <table><tr><td> PROBLEM Related content belongs at one level. </td><td> TRIGGER Tap Bookmarks. </td><td> STATE CHANGE The selected tab index changes. </td><td> VISIBLE RESULT Bookmarks content replaces Lessons content. </td></tr></table> CONNECTION DefaultTabController ↔ TabBar ↔ TabBarView (same route) MENTAL CHECK Should the system Back button undo a tab switch as a route pop? The selected TabBar index and matching TabBarView child change together. TRY IT Add a third tab and child; keep length, tabs, and children equal.	PROBLEM Related content belongs at one level.	TRIGGER Tap Bookmarks.	STATE CHANGE The selected tab index changes.	VISIBLE RESULT Bookmarks content replaces Lessons content.
PROBLEM Related content belongs at one level.	TRIGGER Tap Bookmarks.	STATE CHANGE The selected tab index changes.	VISIBLE RESULT Bookmarks content replaces Lessons content.		

KEY NOTES

- The controller length, number of tabs, and number of TabBarView children must match.
- Use a custom TabController when code must animate to a tab, listen to index changes, or coordinate additional behavior.
- Tabs represent closely related peer content; do not use them to replace a long multi-step workflow.

7.2. Material 3 NavigationBar with IndexedStack

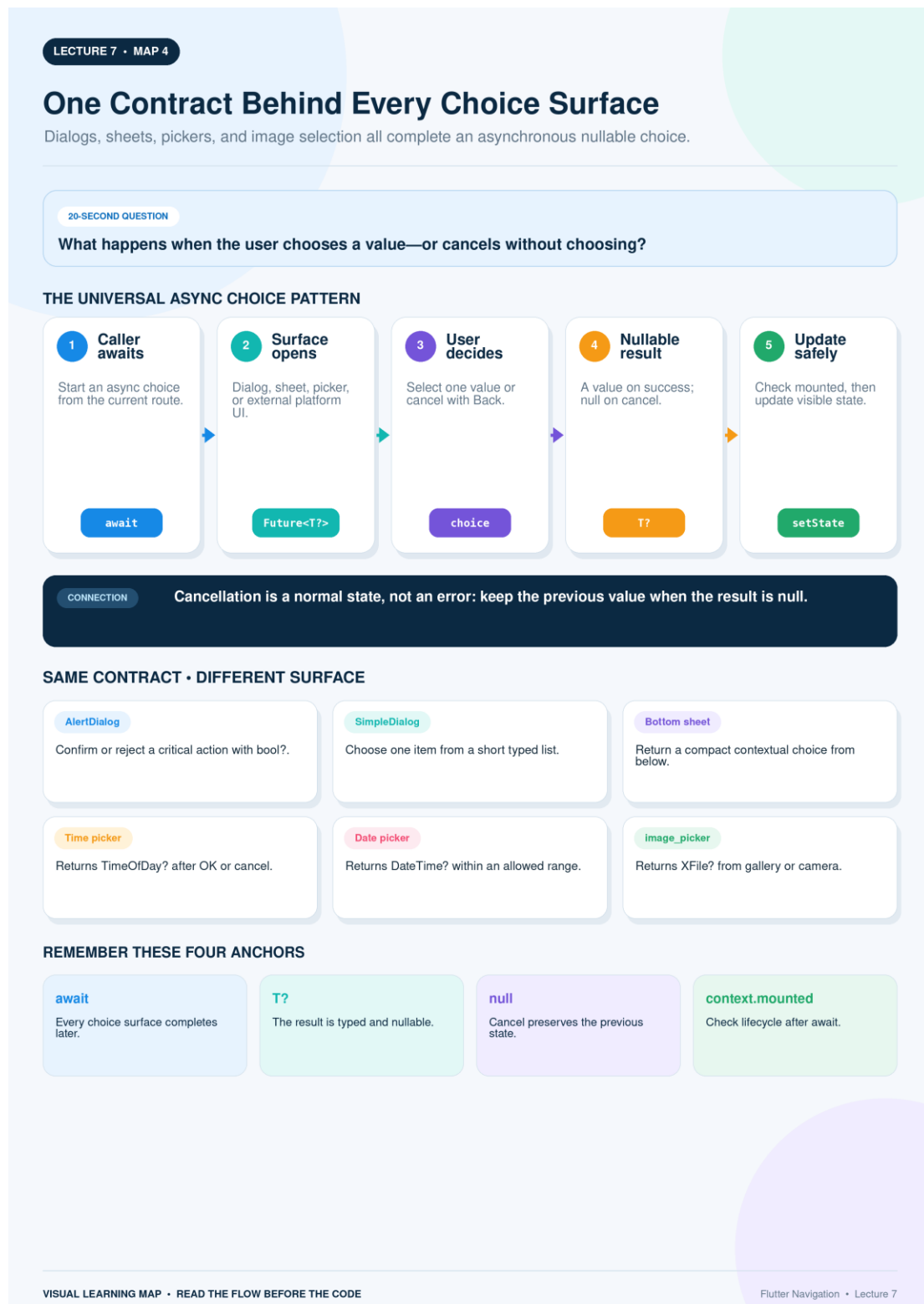
Preserve destination state while switching the selected index

Code	Expected output state(s)				
<pre>INDEXEDSTACK PRESERVES THE TEXTFIELD int index = 0; final searchController = TextEditingController(); late final pages = [const Center(child: Text('Home')), Padding(padding: const EdgeInsets.all(16), child: TextField(controller: searchController, decoration: const InputDecoration(labelText: 'Search'),),), const Center(child: Text('Saved')),]; return Scaffold(body: SafeArea(child: IndexedStack(index: index, children: pages),), bottomNavigationBar: NavigationBar(selectedIndex: index, onDestinationSelected: (value) => setState(() => index = value), destinations: const [NavigationDestination(icon: Icon(Icons.home_outlined), label: 'Home'),</pre>	EXPECTED OUTPUT Preserve destination state with IndexedStack NavigationBar switches top-level destinations while each inactive child keeps its state. 1 TYPE 2 SWITCH 3 RETURN  READ THE FLOW IN 20 SECONDS <table><tr><td>PROBLEM Search text must survive destination switches.</td><td>TRIGGER Type, switch away, then return.</td><td>STATE CHANGE IndexedStack keeps inactive children alive.</td><td>VISIBLE RESULT The word navigation is still present.</td></tr></table> CONNECTION NavigationBar index → IndexedStack index → preserved child state MENTAL CHECK Which widget would rebuild and lose the field if the child were discarded?	PROBLEM Search text must survive destination switches.	TRIGGER Type, switch away, then return.	STATE CHANGE IndexedStack keeps inactive children alive.	VISIBLE RESULT The word navigation is still present.
PROBLEM Search text must survive destination switches.	TRIGGER Type, switch away, then return.	STATE CHANGE IndexedStack keeps inactive children alive.	VISIBLE RESULT The word navigation is still present.		

Code	Expected output state(s)
<pre>NavigationDestination(icon: Icon(Icons.search), label: 'Search'), NavigationDestination(icon: Icon(Icons.bookmark_outline), label: 'Saved'),],),); // Dispose searchController in State.dispose().</pre>	<i>IndexedStack keeps the Search TextField alive, so the typed query remains after switching away and back.</i> TRY IT Replace IndexedStack with pages[index] and compare the preserved value.

8. Dialog Routes and Returned Choices

Await modal routes just like ordinary pushed routes



API	Purpose	Typical return type
showDialog + AlertDialog	Confirm, warn, or request a small decision.	bool?, enum?, or custom value
showDialog + SimpleDialog	Choose one option from a short visible list.	String?, enum?, or model
showModalBottomSheet	Present actions or content from the bottom edge.	Any typed result
Navigator.pop(dialogContext, result)	Close the modal route and complete itsFuture.	Must match the awaited generic type
show ModalBottomSheet<T>	Present a short action or selection surface from the bottom.	T? typed result

8.1. AlertDialog Confirmation

Return true only when the destructive action is confirmed

Code	Expected output state(s)
<pre> AWAIT A BOOLEAN FROM ALERTDIALOG Future<void> confirmDelete(BuildContext context) async { final confirmed = await showDialog<bool>(context: context, barrierDismissible: false, builder: (dialogContext) => AlertDialog(title: const Text('Delete account?'), content: const Text('This action cannot be undone.'), actions: [TextButton(onPressed: () => Navigator.pop(dialogContext, false), child: const Text('Cancel'),), FilledButton(onPressed: () => Navigator.pop(dialogContext, true), child: const Text('Delete'),),],),); if (confirmed == true && context.mounted) { ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Account deleted')),); } } FilledButton(onPressed: () => confirmDelete(context), child: const Text('Delete account'),); </pre>	EXPECTED OUTPUT Confirm a critical action with AlertDialog A modal route returns bool? so the caller can distinguish confirm, reject, and cancel. READ THE FLOW IN 20 SECONDS PROBLEM: Deletion must not happen accidentally. TRIGGER: Tap Delete account. STATE CHANGE: The dialog returns true only on DELETE. VISIBLE RESULT: The caller performs deletion and confirms it. CONNECTION: showDialog<bool> → true / false / null → caller decision MENTAL CHECK: What should happen when Back dismisses the dialog and returns null? Delete returns true and shows Account deleted. Cancel returns false and shows no SnackBar. TRY IT Allow barrier dismissal and verify that tapping outside returns null.

8.2. SimpleDialog Option Selection

Represent a short list of mutually exclusive options

Code	Expected output state(s)
<pre> AWAIT A STRING FROM SIMPLEDIALOG Future<void> chooseLanguage(BuildContext context) async { final language = await showDialog<String>(context: context, builder: (dialogContext) => SimpleDialog(title: const Text('Choose language'), children: [for (final value in ['English', 'Arabic', 'French']) SimpleDialogOption(onPressed: () => Navigator.pop(dialogContext, value), child: Text(value),),],),); if (language == null !context.mounted) return; ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Language: \$language')),); } FilledButton(onPressed: () => chooseLanguage(context), child: const Text('Choose language'),); </pre>	The three options come directly from the code. Selecting Arabic returns it and shows Language: Arabic. TRY IT Replace String with an enum to constrain the result type.

8.3. Modal Bottom Sheet with a Typed Result

Return a compact choice without replacing the entire screen

Code	Expected output state(s)
<pre> AWAIT A TYPED BOTTOM-SHEET RESULT enum SortType { newest, rating, duration } const sortOptions = <SortType, String>{ SortType.newest: 'Newest first', SortType.rating: 'Highest rating', SortType.duration: 'Shortest duration', }; Future<void> chooseSort(BuildContext context) async { final sort = await showModalBottomSheet<SortType>(context: context, builder: (sheetContext) => SafeArea(child: Column(mainAxisAlignment: MainAxisAlignment.min, children: [const ListTile(title: Text('Sort courses')), for (final entry in sortOptions.entries) ListTile(title: Text(entry.value), onTap: () => Navigator.pop(sheetContext, entry.key,),),],),),); if (sort == null !context.mounted) return; ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Sort: \${sort.name}')),); } </pre>	The modal is attached to the bottom edge. Highest rating returns SortType.rating.

Code	Expected output state(s)
<pre>); } FilledButton(onPressed: () => chooseSort(context), child: const Text('Choose sort'),); </pre>	

9. Date and Time Pickers

Await a nullable value, then update state only after a selection

9.1. showTimePicker

Return `TimeOfDay?` and preserve the current value on cancellation

Code	Expected output state(s)
<pre> TIME PICKER STATE + CALLBACK TimeOfDay? selectedTime; Future<void> pickTime() async { final value = await showTimePicker(context: context, initialTime: selectedTime ?? const TimeOfDay(hour: 9, minute: 30), helpText: 'Select reminder time',); if (value == null !mounted) return; setState(() => selectedTime = value); } final label = selectedTime == null ? 'Not selected' : selectedTime!.format(context); return Column(children: [Text('Reminder time: \$label'), FilledButton(onPressed: pickTime, child: const Text('Choose time'),),],); </pre>	EXPECTED OUTPUT Choose a time and update the UI showTimePicker returns TimeOfDay?, only a non-null value replaces the current reminder. BEFORE TIME PICKER AFTER OK READ THE FLOW IN 20 SECONDS PROBLEM The reminder time is not set. TRIGGER Open the picker and press OK. STATE CHANGE TimeOfDay(9, 30) is returned. VISIBLE RESULT The label updates to 09:30. CORRECTION await showTimePicker() -- TimeOfDay? -- setState when non-null MENTAL CHECK If the picker is cancelled, which previous value should remain? The build fragment shows the current label and Choose time. After OK, the locale-formatted selected time replaces Not selected. TRY IT Cancel the picker and verify that the previous selectedTime is preserved.

9.2. showDatePicker

Constrain the selectable range with firstDate and lastDate

Code	Expected output state(s)
<pre> DATE PICKER STATE + CALLBACK DateTime? selectedDate; Future<void> pickDate() async { final value = await showDatePicker(context: context, initialDate: selectedDate ?? DateTime(2026, 7, 24), firstDate: DateTime(2025), lastDate: DateTime(2027, 12, 31), helpText: 'Select course date',); if (value == null !mounted) return; setState(() => selectedDate = value); } final label = selectedDate == null ? 'Not selected' : '\${selectedDate!.year}-\${ '\${selectedDate!.month.toString().padLeft(2, '0')}'- '\${selectedDate!.day.toString().padLeft(2, '0')}}'; return Column(children: [Text('Course date: \$label'), FilledButton(onPressed: pickDate, child: const Text('Choose date'),),],); </pre>	EXPECTED OUTPUT Choose a constrained calendar date showDatePicker returns DateTime? inside firstDate and lastDate; cancellation preserves state. BEFORE DATE PICKER AFTER OK READ THE FLOW IN 20 SECONDS PROBLEM: The course date is not set. TRIGGER: Select Jul 24 and press OK. STATE CHANGE: A valid DateTime is returned. VISIBLE RESULT: The formatted date becomes visible. CONNECTION: showDatePicker --> DateTime? --> validate range --> update UI MENTAL CHECK: Which parameters prevent selection outside the allowed course dates? Selecting 24 July 2026 displays 2026-07-24; cancellation preserves the previous value.

COMMON MISTAKES

- Providing an initialDate outside the allowed range.
- Updating state before checking whether the returned picker value is null.
- Displaying raw DateTime.toString() when the UI needs a localized or deliberately formatted date.

10. Image Selection with image_picker

Use the current XFile API for gallery and camera input

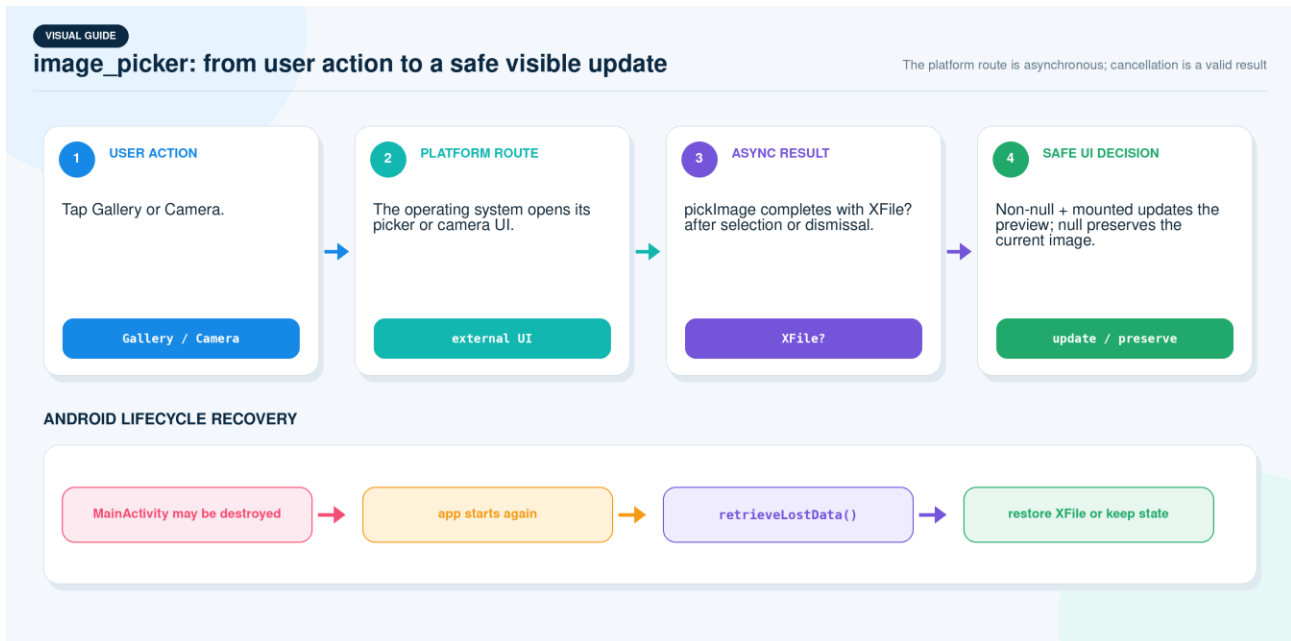
10.1. Package and Platform Setup

External plugin requirements are part of the working example

Area	Required setup
pubspec.yaml	image_picker: ^1.2.3
Import	import 'package:image_picker/image_picker.dart';
Current API	pickImage returns Future<XFile?>; getImage and PickedFile were removed in 1.0.0.
iOS	Add NSPhotoLibraryUsageDescription and NSCameraUsageDescription to ios/Runner/Info.plist.
Android	No Android configuration is required. Do not add requestLegacyExternalStorage; image_picker uses scoped storage and Android 13+ uses Android Photo Picker.
Lifecycle	On Android, run retrieveLostData() at startup to recover picker results after MainActivity destruction.

QUICK IDEA

- ImagePicker opens a platform route outside the Flutter widget tree. The Future completes with an XFile when the user selects/captures media, or null when the user cancels.
- Image.file is suitable for the mobile dart:io example below; web code must render selected bytes or use a web-compatible approach.



10.2. Pick from Gallery or Camera

One method handles both sources and updates one visible image state

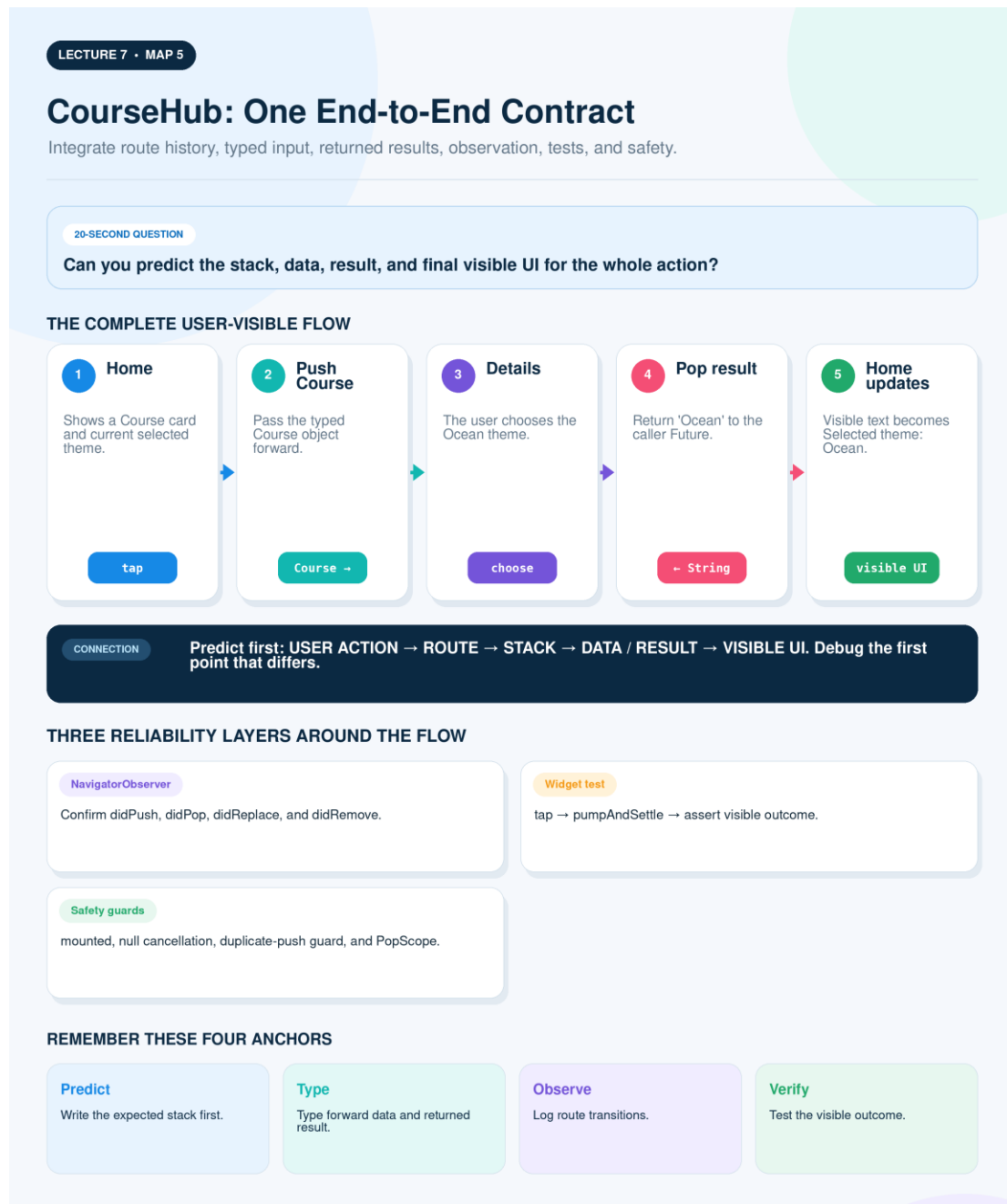
Code	Expected output state(s)
<pre> XFILE STATE + ONE METHOD FOR TWO SOURCES import 'dart:io'; import 'package:image_picker/image_picker.dart'; final picker = ImagePicker(); XFile? image; Future<void> pick(ImageSource source) async { final selected = await picker.pickImage(source: source, imageQuality: 85, maxWidth: 1600,); if (selected == null !mounted) return; setState(() => image = selected); } return Scaffold(appBar: AppBar(title: const Text('Profile photo')), body: Column(children: [Expanded(child: Center(child: image == null ? const Text('No image selected') : Image.file(File(image!.path)),),), Padding(padding: const EdgeInsets.only(bottom: 24), child: Wrap(spacing: 12, children: [OutlinedButton(onPressed: () => pick(ImageSource.gallery), child: const Text('Gallery'),), FilledButton(onPressed: () => pick(ImageSource.camera), child: const Text('Camera'),),],),),],),); </pre>	EXPECTED OUTPUT Pick an image from gallery or camera image_picker returns XFile?; a real selection updates the preview while null means no change. INITIAL STATE AFTER SELECTION READ THE FLOW IN 20 SECONDS PROBLEM The profile has no photo. TRIGGER Choose Gallery or Camera. STATE CHANGE The picker returns a non-null XFile. VISIBLE RESULT The selected image appears in the preview. CONNECTION ImagePicker.pickImage() → XFile? → preview when non-null MENTAL CHECK What should the UI do when the user cancels the external picker? The screen contains only the preview and Gallery/Camera buttons. The selected-image content is illustrative; no extra control is implied. TRY IT Cancel the picker and confirm that the previous image remains unchanged.

COMMON MISTAKES

- Copying outdated getImage/PickedFile examples from old tutorials.
- Assuming the camera exists on every desktop platform without a camera delegate or platform-specific UI.
- Ignoring image size, compression, cancellation, permissions, and lost-data recovery.

11. Integrated Navigation Lab, Observation, and Debugging

Connect the complete workflow, observe route events, and verify behavior with tests



11.1. Complete CourseHub Mini App

Copy the complete code into main.dart: constructor data moves forward, a typed String returns through pop(result), and the caller updates visibly.

COMPLETE EXAMPLE — Unlike the focused snippets above, the following three code blocks form one runnable application.

Complete code — Part 1 of 3: app, model, and observer

```
import 'package:flutter/material.dart';

void main() => runApp(const CourseHubApp());

class CourseHubApp extends StatelessWidget {
  const CourseHubApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(useMaterial3: true),
    navigatorObservers: [AppNavigatorObserver()],
    home: const HomeScreen(),
  );
}

class Course {
  const Course({required this.title, required this.summary});
  final String title;
  final String summary;
}

const demoCourse = Course(
  title: 'Flutter Navigation',
  summary: 'Routes, data flow, and returned values.',
);

class AppNavigatorObserver extends NavigatorObserver {
  @override
  void didPush(Route<dynamic> route, Route<dynamic>? previous) {
    debugPrint('Pushed: ${route.settings.name}');
  }

  @override
  void didPop(Route<dynamic> route, Route<dynamic>? previous) {
    debugPrint('Popped: ${route.settings.name}');
  }
}
```

Complete code — Part 2 of 3: caller route and typed await

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  String selectedTheme = 'None';

  Future<void> openCourse() async {
    final result = await Navigator.push<String>(
      context,
      MaterialPageRoute<String>(
        settings: const RouteSettings(name: '/course-details'),
        builder: (_) => const CourseDetailsScreen(demoCourse),
      ),
    );
  }

  if (result == null || !mounted) return;
  setState(() => selectedTheme = result);
}

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('CourseHub')),
  body: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Text('Selected theme: $selectedTheme'),
        const SizedBox(height: 16),
        Card(
          child: ListTile(
            title: const Text(demoCourse.title),
```

Complete code — Part 2 of 3: caller route and typed await

```

        subtitle: const Text(demoCourse.summary),
        onTap: openCourse,
      ),
    ),
  ],
),
);
}

```

Complete code — Part 3 of 3: destination and pop(result)

```

class CourseDetailsScreen extends StatelessWidget {
  const CourseDetailsScreen(this.course, {super.key});
  final Course course;

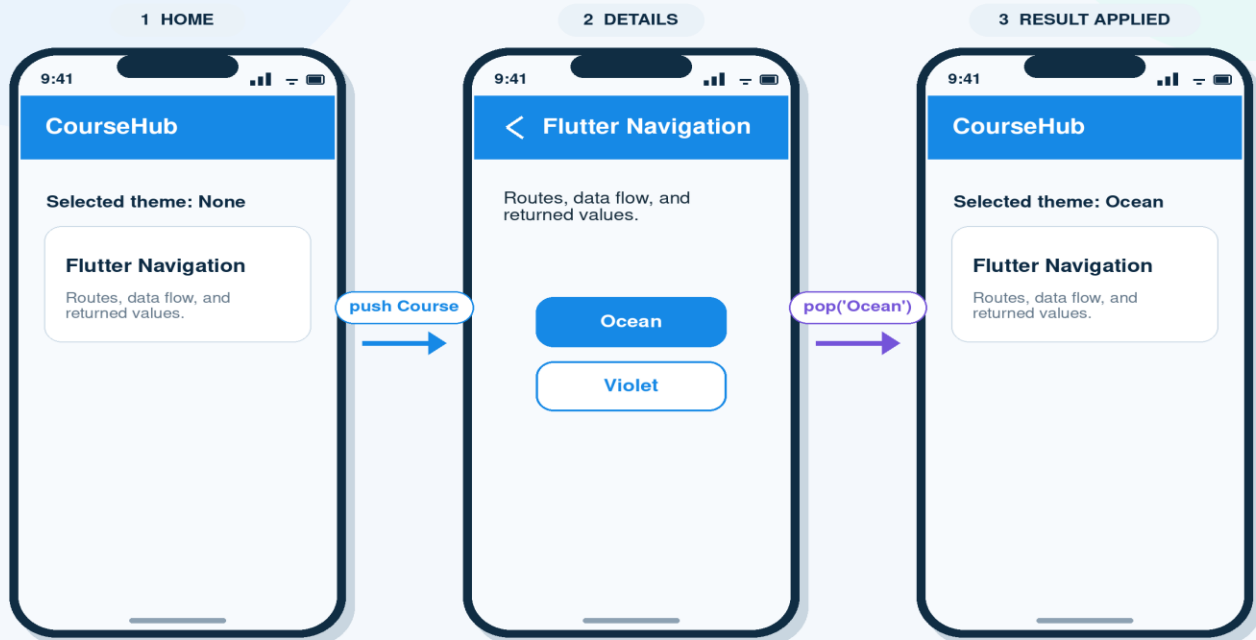
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text(course.title)),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Text(course.summary),
          const SizedBox(height: 24),
          FilledButton(
            onPressed: () => Navigator.pop(context, 'Ocean'),
            child: const Text('Ocean'),
          ),
          const SizedBox(height: 12),
          OutlinedButton(
            onPressed: () => Navigator.pop(context, 'Violet'),
            child: const Text('Violet'),
          ),
        ],
      ),
    ),
  );
}

```

EXPECTED OUTPUT

Complete the CourseHub navigation contract

One flow combines typed input, a returned result, route observation, and visible state.



READ THE FLOW IN 20 SECONDS

PROBLEM

Home needs a theme selected in Details.

TRIGGER

Tap the Course, then choose Ocean.

STATE CHANGE

Course moves forward; Ocean returns back.

VISIBLE RESULT

Home now shows Selected theme: Ocean.

CONNECTION

Course → push → Details → pop(String) → Home setState

MENTAL CHECK

Can you predict the route stack, data, result, and final visible text?

Run sequence: Home → tap Flutter Navigation → choose Ocean → Home displays Selected theme: Ocean.

11.2. Observe Route Changes with NavigatorObserver

Turn route history into readable debug logs and analytics events

Code

OBSERVER CONNECTION USED BY COURSEHUB

```
// MaterialApp:  
navigatorObservers: [AppNavigatorObserver()],  
  
// MaterialPageRoute in openCourse():  
settings: const RouteSettings(name: '/course-details'),  
  
// AppNavigatorObserver is defined in Part 1 above.
```

VISUAL GUIDE

NavigatorObserver makes route history observable

Readable logs turn navigation into evidence

WHAT THE OBSERVER TELLS YOU

- ROUTE HISTORY**: Which route was pushed, popped, replaced, or removed.
- NOT SCREEN DATA**: Keep Course objects and returned values in their typed contracts.
- WHY IT MATTERS**: Use the events for debugging, analytics, and navigation-focused tests.

11.3. Verify Navigation with a Widget Test

Test the exact **CourseHub** action, destination, returned value, and visible **Home** update instead of trusting a static screenshot

Code

```
COURSEHUB WIDGET TEST
import 'package:flutter_test/flutter_test.dart';
// Match this package name to the name in pubspec.yaml.
import 'package:course_hub/main.dart';

void main() {
  testWidgets('returns a selected theme to Home', (tester) async {
    await tester.pumpWidget(const CourseHubApp());

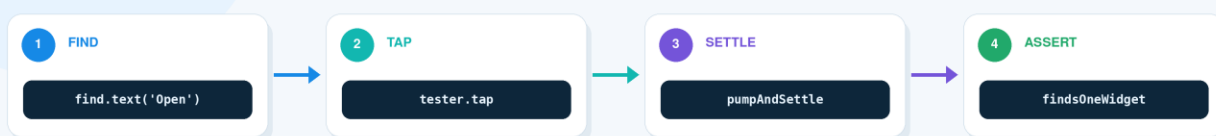
    await tester.tap(find.text('Flutter Navigation'));
    await tester.pumpAndSettle();
    expect(find.text('Routes, data flow, and returned values.'),
      findsOneWidget);

    await tester.tap(find.text('Ocean'));
    await tester.pumpAndSettle();
    expect(find.text('Selected theme: Ocean'), findsOneWidget);
  });
}
```

VISUAL GUIDE

Widget test: verify behavior, not a static screenshot

Action → route transition → visible assertion



11.4. Integration Checklist

Question	What to verify
What action occurred?	Tap, submit, system Back, tab selection, dialog choice, or picker completion.
Which API changed navigation?	push, pop, replacement, reset, tab index, showDialog, or platform picker.
What remains in history?	Predict every route below the current one and whether Back should reach it.
What data moved forward?	Constructor parameter, RouteSettings.arguments, or named-route argument.
What result moved backward?	The typed value supplied to Navigator.pop(result), or null after cancellation.
Is context still valid?	After await, verify mounted/context.mounted before setState, Navigator, or ScaffoldMessenger.

Question	What to verify
What did the user see?	For focused snippets: only the displayed code states. For Section 11.1: the complete Home → Details → returned theme flow.

11.5. Symptom-to-Cause Troubleshooting Matrix

Symptom	Likely cause	Correction
Button does nothing	onPressed is null, not connected, or throws before push.	Break inside the callback and inspect the first failing line.
Navigator not found	The BuildContext is above MaterialApp/Navigator.	Use a descendant context or move the callback below the navigator.
Back returns to login	Login was pushed instead of replaced/reset.	Use pushReplacement or pushAndRemoveUntil for the intended history.
Returned value is always null	Destination pops without a result or user cancels.	Call pop(context, value) and handle cancellation separately.
setState after dispose	Async navigation/picker completed after widget removal.	Check mounted before setState; cancel listeners where applicable.
Hero does not animate	Tags differ or duplicate tags exist on one route.	Use the same unique tag on source and destination.
Tabs mismatch assertion	Controller length differs from tabs or views.	Make all three counts identical.
Image picker code fails	Outdated API or missing platform setup.	Use XFile/pickImage and current package instructions.

REMEMBER

- A reliable navigation feature is a state machine: current route + user action + stack operation + data/result + visible feedback.
- Debug the stack and the Future result, not only the destination widget.

12. Recap and Assessment

Confirm the mental model before memorizing API names

12.1. Decision Guide

Requirement	Preferred starting point
Open a detail page and keep Back	Navigator.push + MaterialPageRoute
Replace login or splash	pushReplacement
Reset a completed/authentication flow	pushAndRemoveUntil
Send typed data	Destination constructor
Return a choice	await push<T> + pop<T>(result)
Switch peer content at the top	TabBar + TabBarView
Switch primary destinations at bottom	NavigationBar + IndexedStack/state solution
Ask for confirmation	showDialog<bool> + AlertDialog
Choose one short option	showDialog<T> + SimpleDialog
Choose a date/time	showDatePicker / showTimePicker
Advanced deep links and web URLs	Router or go_router rather than named routes
Choose a short option from the bottom	showModalBottomSheet<T>
Observe route events	NavigatorObserver
Verify a route flow	Widget test + pumpAndSettle
Custom full-page animation	PageRouteBuilder
Advanced URLs and nested routing	Router or go_router

12.2. Assessment

A. Predict the stack

1. Home → push Courses → push Details → pop. Write the final route stack.
2. Login → pushReplacement Home. What happens when the user presses Back?

B. Predict the returned value

3. What completes await Navigator.push<String> after Navigator.pop(context, 'Dark')?
4. What is returned when a dialog, picker, or bottom sheet is dismissed without a value?

C. Fix the code

- Correct: final value = Navigator.push<String>(context, route);
- Explain why RouteSettingsCourseScreen must read settings.arguments and why the caller must supply a Course.

D. Design the flow

- Build Login → Home → Course Details. Login must not reappear through Back. Choose the operation.
- A Search destination must preserve typed text after switching destinations. Choose the body widget.
- Unsaved profile edits require confirmation on Back. Name the widget and the duplicate-dialog guard.
- Write the essential widget-test steps for CourseHub → Flutter Navigation → Ocean → returned Home value.

FINAL CHECK

- Can the student predict the route stack before every action?
- Do focused outputs contain only elements created by their snippets, and does the complete example run as one application?
- Are cancellation, nullable results, mounted checks, and back behavior explicitly handled?

12.3. Official References

- Flutter navigation and routing — <https://docs.flutter.dev/ui/navigation>
- Navigate to a new screen and back — <https://docs.flutter.dev/cookbook/navigation/navigation-basics>
- Send data to a new screen — <https://docs.flutter.dev/cookbook/navigation/passing-data>
- Return data from a screen — <https://docs.flutter.dev/cookbook/navigation/returning-data>
- Named routes — <https://docs.flutter.dev/cookbook/navigation/named-routes>
- Hero animations — <https://docs.flutter.dev/cookbook/navigation/hero-animations>
- PopScope API — <https://api.flutter.dev/flutter/widgets/PopScope-class.html>
- TabBar API — <https://api.flutter.dev/flutter/material/TabBar-class.html>
- Material dialogs — <https://api.flutter.dev/flutter/material/AlertDialog-class.html>
- showModalBottomSheet API — <https://api.flutter.dev/flutter/material/showModalBottomSheet.html>
- NavigatorObserver API — <https://api.flutter.dev/flutter/widgets/NavigatorObserver-class.html>
- Widget testing introduction — <https://docs.flutter.dev/cookbook/testing/widget/introduction>
- image_picker package — https://pub.dev/packages/image_picker
- go_router package — https://pub.dev/packages/go_router

13. Applied Project Pack: CourseHub+

A complete assessment-ready extension of the integrated navigation lab

PROJECT OBJECTIVE

- Create a Material 3 application that demonstrates correct route history, typed data transfer, typed results, modal choices, state-preserving destinations, back protection, observation, and widget testing.
- The project must compile without deprecated Flutter navigation or image_picker APIs.
- Every feature must include a visible outcome that can be demonstrated or asserted in a test.

13.1. Required Functional Scope

Requirement	Acceptance evidence
Authentication flow	Login opens Home with replacement or reset; Back never reveals Login.
Course navigation	Home opens Course Details using a typed Course model.
Returned value	Details returns Ocean or Violet and Home updates the selected theme label.
Primary destinations	Home, Search, and Saved use NavigationBar; Search text is preserved.
Modal choice	A typed bottom sheet returns a SortType value; cancellation preserves the old value.
Unsaved form	Edit Profile uses PopScope and prevents duplicate confirmation dialogs.
Media selection	Profile image uses image_picker 1.2.3 with XFile and cancellation handling.

Observation	NavigatorObserver logs push, pop, and replacement route names.
Verification	At least two widget tests verify the main result flow and cancellation/Back behavior.

SUBMISSION FILES

- Source code under lib/ with clear feature responsibilities.
- pubspec.yaml and test/course_hub_navigation_test.dart.
- A one-page navigation decision log explaining every push, replacement, reset, modal, and result type.
- Screenshots or a short recording of the required flows.

13.2. Project Architecture and Milestones

Build the project in verifiable increments rather than one large implementation

RECOMMENDED FILE STRUCTURE

```
lib/
  main.dart
  app.dart
  models/course.dart
  models/theme_choice.dart
  navigation/app_navigator_observer.dart
  screens/login_screen.dart
  screens/home_shell.dart
  screens/course_details_screen.dart
  screens/search_screen.dart
  screens/edit_profile_screen.dart
  services/profile_image_service.dart

test/
  course_hub_navigation_test.dart
```

Milestone	Core task	Evidence
M1 — Stack	Login → Home uses replacement/reset.	Back does not reveal Login.
M2 — Typed flow	Pass Course; return ThemeChoice.	Home label updates after result.
M3 — Destinations	NavigationBar + IndexedStack.	Search query remains after switching.
M4 — Modal	Return SortType from bottom sheet.	Selected sort is visible; cancel is safe.
M5 — Back guard	PopScope protects dirty profile form.	Stay/Discard behavior is correct.
M6 — Media	Pick XFile from gallery/camera.	Preview updates; cancel keeps old image.
M7 — Tests	Automate main flow and cancellation.	All widget tests pass.
M8 — Review	Run troubleshooting checklist.	No duplicate push or stale-context warning.

TEAM WORKFLOW

- Developer A implements a milestone while Developer B writes its acceptance test and route-stack prediction.
- Swap roles at the next milestone.
- Do not merge a milestone until the visible outcome and the test agree with the predicted history.

13.3. Project Test Cases and Expected Outcomes

Use the same scenarios for manual demonstration and automated verification

ID	Action sequence	Expected outcome
T1	Launch → Login → Continue	Home is visible; Login is absent from Back history.
T2	Tap Flutter Navigation → choose Ocean	Home displays Selected theme: Ocean.
T3	Open Details → system Back	Theme remains unchanged because result is null.
T4	Type "routing" in Search → Home → Search	The query "routing" is still visible.
T5	Open Sort sheet → Highest rating	SortType.rating is returned and shown.
T6	Open Sort sheet → dismiss	Previous sort remains unchanged.
T7	Edit profile → change field → Back → Stay	Edit screen remains open with changes.
T8	Edit profile → Back → Discard	Edit screen closes once; no duplicate dialog appears.
T9	Gallery → cancel	Existing image/placeholder remains unchanged.
T10	Open and close Course Details	Observer prints named PUSH and POP events.

CANCELLATION/BACK WIDGET TEST

```
testWidgets('Back returns null and preserves the theme', (tester) async {
  await tester.pumpWidget(const CourseHubApp());
  await tester.tap(find.text('Flutter Navigation'));
  await tester.pumpAndSettle();

  await tester.pageBack();
  await tester.pumpAndSettle();

  expect(find.text('Selected theme: None'), findsOneWidget);
});
```

PASS CONDITION

- The test proves that cancellation is a designed state, not an error path.
- The caller updates only after a non-null typed result and a valid mounted check.

13.4. Project Rubric and Answer Guide

Evaluate navigation correctness before visual decoration

Criterion	Weight	Full-credit evidence
Route history and Back behavior	20%	Correct use of push, replacement, reset, and modal routes.
Typed data and result contracts	15%	Course and enum values move with matching generic types.
Destination state preservation	10%	Search state survives destination switching.
Cancellation and mounted safety	10%	Null is handled; no setState-after-dispose warning.
PopScope protection	10%	Dirty state, Stay/Discard, and duplicate-dialog guard work.
Current image_picker integration	10%	XFile, permissions, cancellation, and lifecycle notes are correct.
Observation and troubleshooting	5%	Named route events and diagnostic reasoning are shown.
Widget tests	15%	Main flow and cancellation/Back tests pass.
Code organization and explanation	5%	Readable files and a precise decision log.

Answer guide for the lecture assessment

- Home → Courses → Details → pop leaves [Home, Courses].
- pushReplacement from Login to Home prevents Back from revealing Login.
- Navigator.pop(context, "Dark") completes await Navigator.push<String> with "Dark".
- Dismissal without a value returns null.
- Correct syntax: final value = await Navigator.push<String>(context, route);
- Completed checkout normally uses pushAndRemoveUntil with a suitable predicate.
- Search state can be preserved with IndexedStack or a deliberate external state solution.
- Unsaved work uses PopScope plus a guard such as dialogOpen.
- Named-route arguments are dynamic and must be validated before casting.

13.5. Final Project Checklist and Updated References

Release the project only when the behavior, stack, and tests agree

Final check	Yes / No
Can every callback be connected to one explicit user action?	
Is the selected API the smallest one that leaves the correct Back history?	
Are all forward values typed and destination requirements explicit?	
Are all returned values typed and nullable cancellation handled?	
Is context.mounted or mounted checked after asynchronous gaps?	
Are duplicate pushes and duplicate confirmation dialogs prevented?	
Do bottom destinations preserve the required state?	
Does image selection use XFile and current platform guidance?	
Do observer logs use stable route names without sensitive data?	
Do the widget tests prove user-visible outcomes?	

VERSION BASELINE — VERIFIED 8 AUGUST 2026

- Flutter stable family: 3.44.x; Dart 3.12.x; Material 3.
- image_picker: 1.2.3.

- `go_router`: 17.3.0.
- Named routes remain educational/legacy-compatible but are not recommended for most applications.

Official references

- Flutter navigation and routing — <https://docs.flutter.dev/ui/navigation>
- Navigation cookbook — <https://docs.flutter.dev/cookbook/navigation>
- `PopScope` — <https://api.flutter.dev/flutter/widgets/PopScope-class.html>
- `NavigatorObserver` — <https://api.flutter.dev/flutter/widgets/NavigatorObserver-class.html>
- Widget testing — <https://docs.flutter.dev/cookbook/testing/widget/introduction>
- `image_picker` — https://pub.dev/packages/image_picker
- `go_router` — https://pub.dev/packages/go_router
- Flutter release notes — <https://docs.flutter.dev/release/release-notes>